



“ Formando líderes para la construcción de un nuevo país en paz ”

LABORATORIO CRUD

Estudiantes:

HELEN NATALIA CABARICO DUARTE - 1094220506

JOHANDER JOSÉ SARMIENTO RAMÍREZ -1127047378

DIDIER ANDRE VESGA FUENTES – 1092529198

DESARROLLO ORIENTADO A PLATAFORMAS

Grupo: Ar

FANNY CASADIEGO CHIQUILLO

26 de noviembre del 2025



INDICE

1. Laboratorio.....	3
2. Solución Taller.....	6
2.1 Capturas de pantalla de las pruebas de los endpoints:	6
2.2 Documento corto respondiendo:	11



1. Laboratorio

Gestión de Estudiantes con Spring Boot

Construcción de un servicio REST para la gestión de estudiantes usando Spring Boot

En este laboratorio, debe implementar un **servicio web REST** utilizando **Spring Boot**, con el fin de gestionar un conjunto de estudiantes en una aplicación backend.

El propósito es reforzar conceptos de controladores REST, modelos de datos, servicios de negocio y manejo de colecciones en memoria.

El servicio permitirá realizar las operaciones básicas de un CRUD (**Crear, Leer, Actualizar y Eliminar**) sin utilizar todavía bases de datos externas.

Los datos serán almacenados en una colección interna para facilitar la comprensión de la arquitectura inicial.

Contexto del ejercicio:

La Universidad desea implementar un módulo básico para registrar estudiantes en un prototipo de plataforma académica. Por ahora, solo se necesita un servicio backend que permita registrar estudiantes y consultar su información a través de rutas HTTP.

Este laboratorio sirve como base para futuros módulos como matrículas, asignaturas o notas.

El objetivo principal es que el estudiante comprenda el flujo entre:



- **Controlador REST**
- **Servicio de lógica de negocio**
- **Modelo de datos**
- **Uso de rutas HTTP (GET, POST, PUT, DELETE)**

Para controlar el modelo se deben usar:

- Crear controladores REST usando `@RestController` y `@RequestMapping`.

`@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@Service`, Manejo de listas en memoria (`ArrayList`).

- Implementar modelos de datos en Java.
- Separar responsabilidades mediante un servicio de negocio (`@Service`).
- Crear endpoints que reciban y respondan datos en formato JSON.
- Usar Postman o navegador para consumir servicios REST.
- Comprender el funcionamiento básico de un CRUD antes de integrar bases de datos.

Requerimientos funcionales:

El sistema debe permitir:

1. **Listar todos los estudiantes (GET).**
2. **Buscar un estudiante por ID (GET).**
3. **Registrar un nuevo estudiante (POST).**
4. **Actualizar los datos de un estudiante existente (PUT).**
5. **Eliminar un estudiante por ID (DELETE).**



Cada estudiante debe contener como mínimo:

- id (entero)
- nombre (cadena de texto)
- edad (entero)

Los datos iniciales deben incluir **al menos dos estudiantes de ejemplo** cargados en memoria.

OJO:

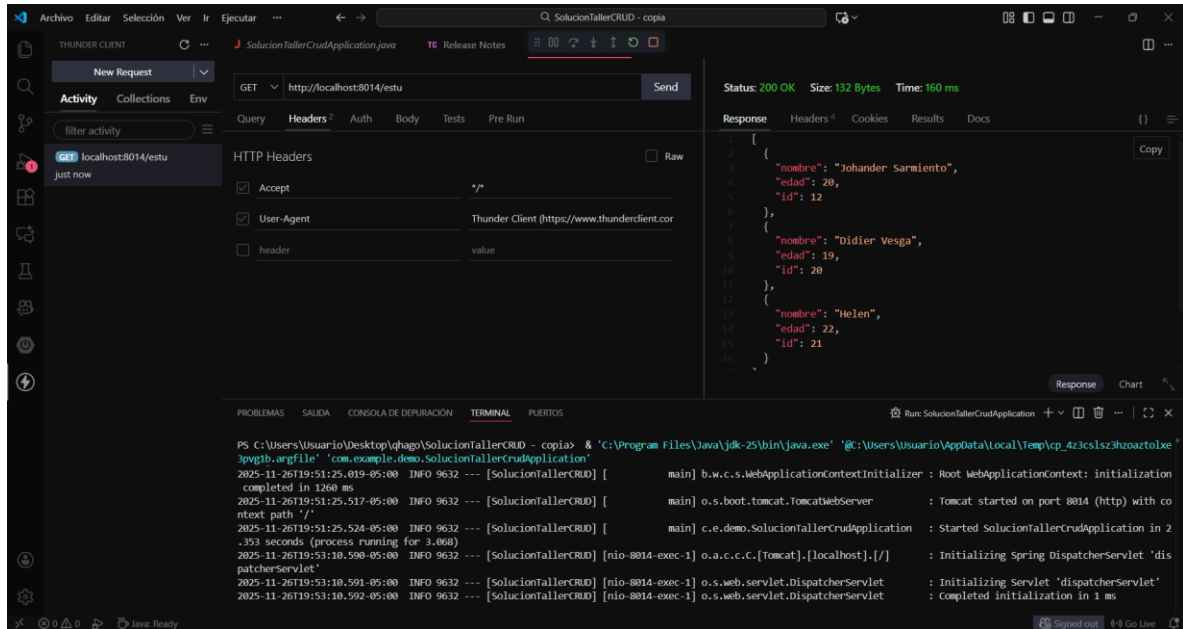
- Código fuente completo del proyecto con las siguientes capas:
model
service
Controller



2. Solución Taller

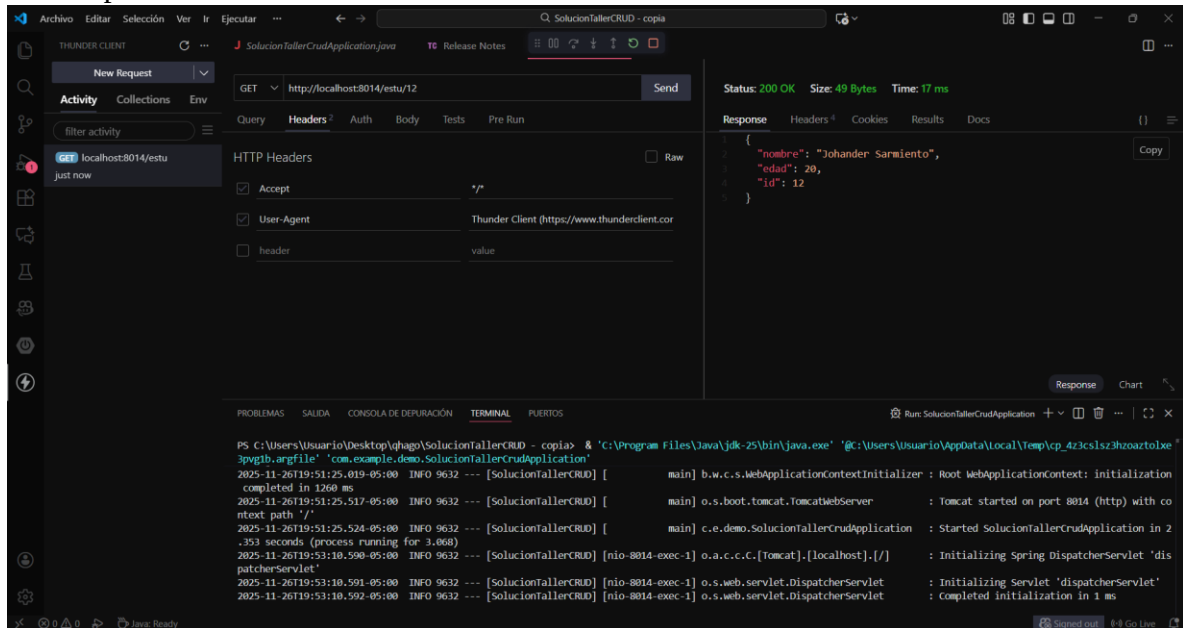
2.1 Capturas de pantalla de las pruebas de los endpoints:

- Listar estudiantes





- Buscar por ID





- Crear estudiante

The first screenshot shows a POST request to `http://localhost:8014/estu` with a JSON body: `{ "nombre": "Javier Paez", "edad": 22, "id": 8 }`. The response is a 200 OK status with a JSON body: `{ "nombre": "Javier Paez", "edad": 22, "id": 8 }`. The terminal shows the application running on port 8014.

The second screenshot shows a GET request to `http://localhost:8014/estu/8` with a JSON body: `{ "nombre": "Javier Paez", "edad": 22, "id": 8 }`. The response is a 200 OK status with a JSON body: `{ "nombre": "Javier Paez", "edad": 22, "id": 8 }`. The terminal shows the application running on port 8014.



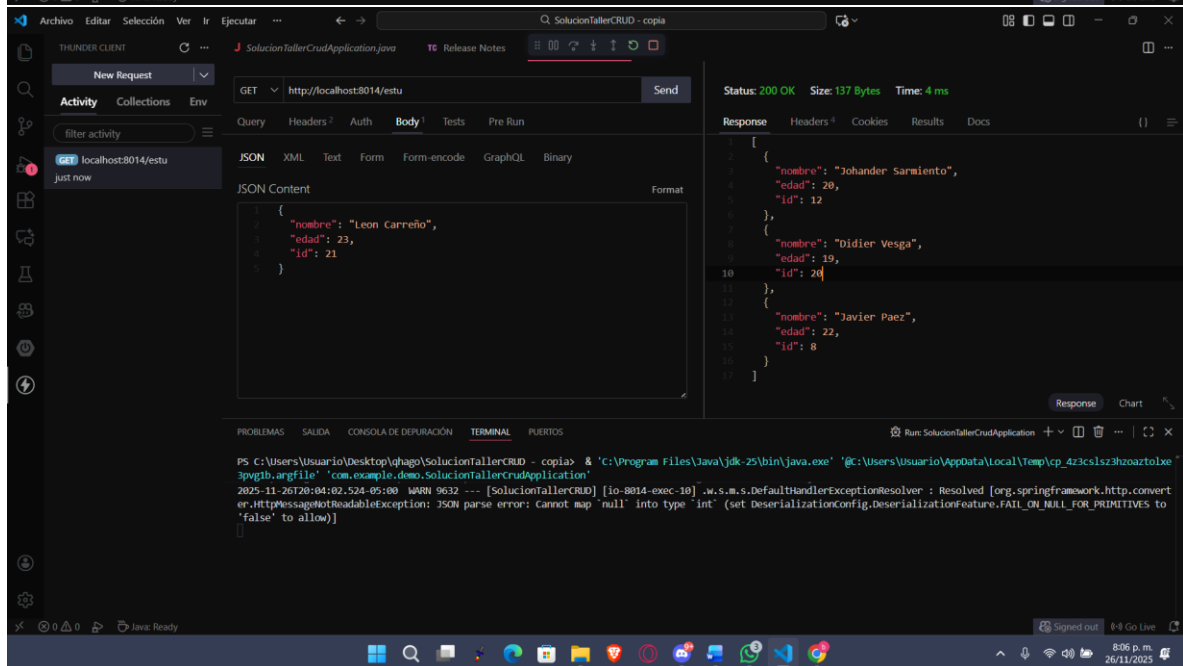
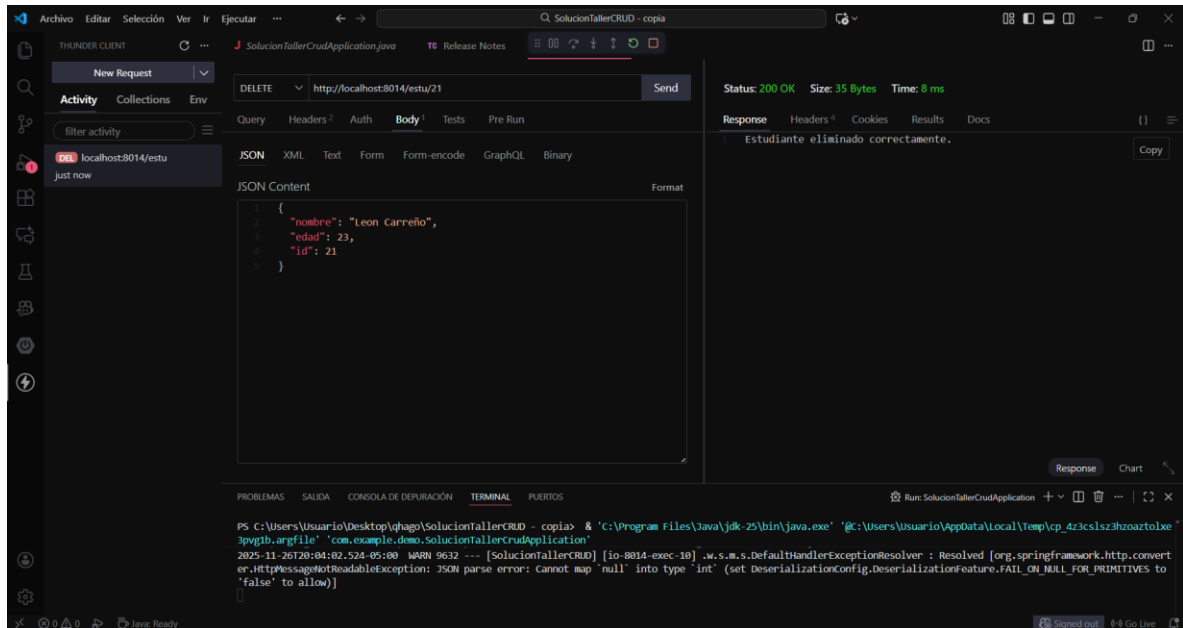
- Actualizar estudiante

Thunder Client interface showing a PUT request to `http://localhost:8014/estu/21`. The response is a JSON object: `{ "nombre": "Leon Carreño", "edad": 23, "id": 21 }`. The terminal shows the application running and logging the update.

Thunder Client interface showing a GET request to `http://localhost:8014/estu`. The response is a JSON array of three student objects: `[ { "nombre": "Leon Carreño", "edad": 23, "id": 21 }, { "nombre": "Johander Sarmiento", "edad": 20, "id": 12 }, { "nombre": "Didier Vesga", "edad": 19, "id": 20 } ]`. The terminal shows the application running and logging the retrieval.



- Eliminar estudiante





2.2 Documento corto respondiendo:

- ¿Qué ventajas y desventajas tiene usar listas en memoria?

RTA: / Usar listas en memoria tiene como ventaja que el acceso a los datos es muy rápido, ya que todo se almacena directamente en la RAM sin necesidad de realizar consultas externas. Esto permite que el desarrollo inicial del CRUD sea más sencillo, ligero y fácil de depurar, ideal para aprender la arquitectura básica entre controlador, servicio y modelo.

Sin embargo, la principal desventaja es que los datos no son persistentes: cuando se reinicia la aplicación, toda la información se pierde. Además, las listas en memoria no están diseñadas para manejar grandes volúmenes de datos ni para garantizar integridad, seguridad o concurrencia, lo cual limita su uso en aplicaciones reales.

- ¿Qué ocurriría si hubiese múltiples usuarios accediendo al sistema al mismo tiempo?

RTA: / Si varios usuarios accedieran al sistema simultáneamente, podrían presentarse problemas de concurrencia porque las listas en memoria no están preparadas para manejar acceso concurrente. Por ejemplo, dos usuarios podrían intentar modificar o eliminar el mismo estudiante al mismo tiempo, generando inconsistencias en los datos. También podrían ocurrir errores como duplicación accidental de IDs o incluso pérdida de información al no existir mecanismos de bloqueo, transacciones o aislamiento. Esto vuelve el sistema inseguro y poco confiable en escenarios multiusuario.

- ¿Qué mejorarías si usaras una base de datos real?

RTA: / Al usar una base de datos real, se obtendría persistencia de los datos incluso después de reiniciar la aplicación. También se mejorarían aspectos como la integridad, la seguridad y el manejo de concurrencia, evitando conflictos cuando muchos usuarios trabajan al mismo tiempo. Además, las bases de datos permiten realizar búsquedas más complejas, manejar grandes volúmenes de información y escalar el sistema de forma profesional. Usar una base de datos también facilitaría



implementar validaciones, llaves primarias, índices, transacciones y otras características importantes en aplicaciones reales.